Import Statements

In [1]:
```
import os
import custom_cmap
import sys
from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce
from pathlib import Path
from astropy.visualization import PercentileInterval

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch,

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory strucutre you need.

00.DATA
01.XYM
02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION
All the necessary scripts are included in the sample data directory. The simplest way to run MORIA on your target is to copy the entirety of the "data" folder here, put your exposures in "data/00.DATA", then beginning with the demo.

After you finish running this notebook, you will be able to use the "_flc" exposures to generate an output image stack in the F814W and F606W HST filters.

NOTE: chmod +x program.src is a useful command to use whenever permissions are denied for a script.

In [2]:
```
#This is the directory where you are processing your data. This does not poi
directory = os.getcwd()
```

Data Preparation

MORIA runs the "Makefile" in fortran_compile directly upon pip installation.

If you have different fortran dependincies, you will have to edit the Makefile.txt in fortran_compile, and then uncomment the code_directory to point to src/fortran_compile. Once you finish compiling the files above, run this cell below to put the fortran scripts in their appropriate directories:

In [ ]:
```
reduce.data_prep_early(directory)
```

STEP 1

Download the _flc files. Place them appropriately in the F814W and F606W directories in 00.DATA.
The cell below will convert the _flc files to _WJ2 files using run_convert_C1K1C.src
These "_flc" exposures include the charge-transfer efficiency (CTE) corrections. MORIA first converts these "_flc" exposures into "_WJ2" files to convert to a full-chip co-ordiante system that are placed into appropriate reference geometry for subsequent steps.

In [7]:
```
reduce.run_xgf_conversion(directory)
```

```
NARG: FILENAME:            1  iffp02bxq_WJ2.fits
NARG: FILENAME:            2  iffp02byq_WJ2.fits
NARG: FILENAME:            3  iffp02c0q_WJ2.fits
NARG: FILENAME:            4  iffp02c1q_WJ2.fits
NARG: FILENAME:            5  iffp02c2q_WJ2.fits
NARG: FILENAME:            6  iffp02c3q_WJ2.fits
NARG: FILENAME:            7  iffp02c4q_WJ2.fits
NARG: FILENAME:            8  iffp02c5q_WJ2.fits
NARG: FILENAME:            9  iffp02c6q_WJ2.fits
NARG: FILENAME:           10  iffp02c7q_WJ2.fits
NARG: FILENAME:            1  iffp02bhq_WJ2.fits
NARG: FILENAME:            2  iffp02biq_WJ2.fits
NARG: FILENAME:            3  iffp02bjq_WJ2.fits
NARG: FILENAME:            4  iffp02bkq_WJ2.fits
NARG: FILENAME:            5  iffp02blq_WJ2.fits
NARG: FILENAME:            6  iffp02bmq_WJ2.fits
NARG: FILENAME:            7  iffp02bnq_WJ2.fits
NARG: FILENAME:            8  iffp02boq_WJ2.fits
NARG: FILENAME:            9  iffp02bpq_WJ2.fits
NARG: FILENAME:           10  iffp02bqq_WJ2.fits
NARG: FILENAME:           11  iffp02brq_WJ2.fits
NARG: FILENAME:           12  iffp02bsq_WJ2.fits
NARG: FILENAME:           13  iffp02btq_WJ2.fits
NARG: FILENAME:           14  iffp02buq_WJ2.fits
NARG: FILENAME:           15  iffp02bvq_WJ2.fits
NARG: FILENAME:           16  iffp02bwq_WJ2.fits
```

Step 2

Prepare the IN.* files taken in by the .src scripts in 01.XYM and 02.CMD directories. This step is important since the IN.* files in each directory are used to provide input to the backend Fortran scripts

In [8]:
```
reduce.data_prep(directory)
```

Creature matchup files

In the cell below we will achieve the following outcomes:

We will use respective PSF files for each filter to create a .XYM file for each exposure. A library PSF model is used to produce ".XYM" files, containing the star positions and magnitudes for each HST image exposure.
Generate output files TRANS.xym2mat, along with 16 MAT.0 files. Note that the first line of the IN.xym2mat and files beginning with 00 just defines the reference coordinates to be used for the output, so the 00 file does not need to be a *_WJ2.xym file.
Find all the objects that could be found in at least 12 out of 16 exposures.

In [11]:
```
reduce.matchup_files(directory)
```

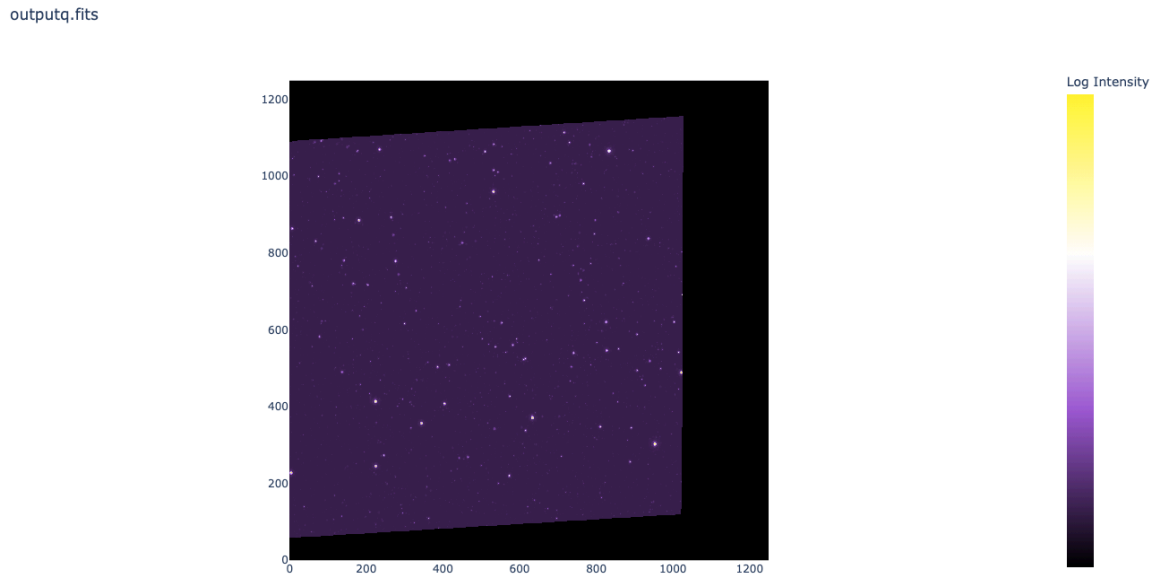
Step 3

Create a file called outpuq.fits. This step creates a stack of the scene in the reference frame. You can use the output fit files to identify your target. This step will prepare the output stack in both filters (F814W and F606W)

In [14]:
```
reduce.run_output_stack(directory)
```

You can view the output stack in the jupyter notebook here. Adjust the scaling accordingly. You can modify the filename to open the output stack in the F606W filter. For more freedom, you may want to open everything via SAO DS9

In [15]:
```
filename = Path(directory).resolve()/f"01.XYM/F814W/outputq_F814W.fits"
#filename = Path(directory).resolve()/f"01.XYM/F814W/outputq_F606W.fits"

hdul = fits.open(filename)
data = hdul[-1].data
hdul.close()
data = np.nan_to_num(data, nan=0.0, posinf=0.0, neginf=0.0)

norm = ImageNormalize(data, stretch=AsinhStretch(0.02))
scaled = norm(data)

nonbinary_colors = custom_cmap.nb_colors()

fig = px.imshow(
    scaled,
    origin='lower',
    color_continuous_scale=nonbinary_colors,
    title="outputq.fits",
    aspect='equal'
)

fig.update_layout(
    width=750,
    height=750,
    coloraxis_colorbar=dict(title="Log Intensity", tickvals=[]),
)
fig.show()
```


outputq.fits
Log Intensity
1200
1000
800
600
400
200
0
0
200
400
600
800
1000
1200

In [ ]: